PukSquash Developer Manual

Registration Payments and Firebase Extension

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

The functional and technical specifications introduce PayFast registration and private-coaching payments in Phase 2. This part describes the Firebase integration boundary and the work required to add those payments to the documented Firebase edition. It is an extension design, not a claim that revision fe3e361 already collects fees.

The later repository contains registration checkout and PayFast callback code. Its business flow can guide the extension: authenticate with Firebase, verify email, create a pending registration and payment attempt, send the user to hosted checkout, validate the server callback, then complete registration idempotently. The Data Connect schema and operations for this flow still need implementation.

1 1 Files and Responsibilities

resources/js/pages/RegistrationPaymentPage.vue, PaymentReturnPage.vue and PaymentCancelPage.vue — later checkout screens

app/Http/Controllers/Api/RegistrationController.php and PayfastItnController.php — later flow reference

app/Services/PayfastService.php and config/payfast.php — gateway logic

dataconnect/schema/schema.gql and connector operations — required Firebase extension

app/Http/Middleware/EnsureRegisteredUser.php — later registration gate to adapt

1 2 API Contract

Later reference routes are GET /api/registration/status, POST /api/registration/payment/initiate, POST /api/registration/reconcile and POST /api/payfast/itn/registration. These are not present in the fe3e361 route set. The ITN callback authenticates the payment notification rather than a browser Firebase token.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Proposed registration state handling

Proposed state flow for the Firebase extension

```
Firebase account created
        -> Email verification pending
Verified Firebase identity
        -> Pending registration
Payment attempt created on the server
        -> Hosted PayFast checkout
Verified COMPLETE notification
        -> Record payment as paid
        -> Create or activate application profile
        -> Mark registration completed
Duplicate notification
        -> Return the same completed result
```

Explanation

Keep identity creation separate from payment completion. A browser return URL is navigation, not payment proof. The backend must compare the callback with the stored payment attempt and validate the gateway notification before recording a successful payment or granting paid access.

Physical Code Explanation

1 Pending registration stores the verified Firebase UID and the profile fields waiting for activation. It must not contain the user's password.

2 A payment attempt has a unique merchant payment reference, server-selected amount in integer cents, currency, status and gateway reference.

3 A verified callback can arrive more than once or before the browser returns. Completion must produce one profile and one paid result.

4 If payment is verified but profile completion fails, retain enough durable state for reconciliation without charging again.

The fee is server configuration, not a browser-controlled amount. Keep PAYFAST_MERCHANT_ID, PAYFAST_MERCHANT_KEY and PAYFAST_PASSPHRASE server-side. Configure return, cancel and notify URLs for the correct environment.

3 Required Data Connect persistence contract

Design contract — names are proposed, not existing GraphQL operations

```
Required new entities
  PendingRegistration
  RegistrationPayment

Required server operations
  ReadRegistrationStatus
  CreateRegistrationPaymentAttempt
  CompleteRegistrationPayment
  ReconcileRegistration

Completion invariants
  Unique merchant payment reference
  Unique provider transaction reference where applicable
  Exact stored amount and currency match
  One application profile per Firebase UID
  Repeat completion has no duplicate side effects
```

Explanation

Add and review the new schema and privileged operations before connecting the later checkout screens to the Firebase edition. Payment completion needs a transaction-safe database operation. Independent calls to SqlConnectClient::execute do not become atomic simply because a Laravel DB::transaction surrounds them.

Physical Code Explanation

1 ReadRegistrationStatus must be scoped to the verified caller. A caller must not inspect another user's payment by supplying their UID.

2 CreateRegistrationPaymentAttempt records the expected fee before checkout so the callback can be checked against a server record.

3 CompleteRegistrationPayment validates the stored state and applies idempotent changes atomically within the persistence layer.

4 ReconcileRegistration safely retries incomplete activation after payment verification; it must not trust a browser assertion that payment succeeded.

Check PayFast signature, merchant, expected amount, allowed payment status and server confirmation, with source validation configured correctly for the deployment. Production must not enable relaxed validation or demo-completion routes. See the official PayFast integration documentation for the exact callback validation rules.

4 Verification and Maintenance

Verification checklist

1 Test verified payment, cancellation, failure and a browser return before callback processing. Access must follow server registration state.

2 Send a duplicate COMPLETE notification and concurrent completion requests. Confirm only one payment result and application profile.

3 Reject wrong signature, merchant, reference and amount without activating the account.

4 Simulate Firebase lookup failure and Data Connect failure after payment verification. Confirm reconciliation can recover without another charge.

5 Adapt RegistrationPaymentInitiationTest and PayfastItnRegistrationTest to the Data Connect repositories and add real connector integration coverage. Their later direct-database mocks are not proof that this extension works.

Implementation boundaries

The Firebase baseline GET /api/me automatically creates a profile. A paid-registration extension must revise that behaviour so profile existence cannot bypass the registration gate. Keep status and checkout routes reachable by a verified but not-yet-registered identity.

Private coaching payments need a separate booking reservation and expiry policy. Registration payment code does not implement coaching checkout, fee ownership or refund handling. Define those contracts before enabling paid coaching.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

Payment flow reference: later repository revision a3cc4c3. The proposed Data Connect entities and operations in this part have not been implemented or tested by this documentation task.

https://developers.payfast.co.za/

https://firebase.google.com/docs/sql-connect/admin-sdk